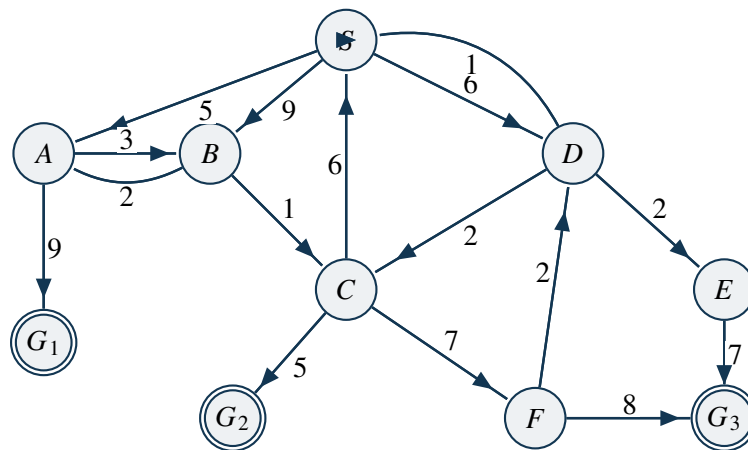


Inteligência Artificial

FORMULAÇÃO DE PROBLEMAS E PROCURA NÃO INFORMADA

QUESTÃO 1 — PROCURA NUM GRAFO: BFS, DFS E UCS

Considere o grafo dirigido e ponderado seguinte. O estado inicial é S e os estados objetivo são G_1 , G_2 e G_3 . Só é permitido percorrer cada ligação no sentido da seta. Os números indicam os custos das ações.



O que se pede

- Execute BFS, DFS e UCS até retirar os três objetivos. Para cada algoritmo, apresente a evolução de OPEN e VISITED, explicando as repetições ignoradas e as atualizações efetuadas.
- Desenhe a árvore de pesquisa no momento em que cada objetivo é retirado. Distinga os nós pendentes dos já visitados e assinale o caminho até ao objetivo desse momento.
- Indique a ordem de retirada dos estados e a ordem de confirmação dos objetivos. Para cada um de G_1 , G_2 e G_3 , reconstrua o caminho encontrado, a profundidade e o custo total.
- Compare os resultados. Distinga a ordem de retirada do caminho solução e explique por que motivo a UCS usa o custo acumulado, e não apenas o custo da última ligação.

Solução proposta

As tabelas apresentam as estruturas depois de cada passo. Na UCS, a notação $n : c$ indica o estado e o seu custo acumulado. VISITED inclui os objetivos retirados, embora estes não sejam expandidos.

Execução da BFS

@p.05p.09p.37p.42@

Passo Retirado OPEN após o passo VISITED

```

0 - [S] []
1 S [A, B, D] [S]
2 A [B, D, G1] [S, A]
3 B [D, G1, C] [S, A, B]
4 D [G1, C, E] [S, A, B, D]
5 G1 [C, E] [S, A, B, D, G1]

```

6 C [E, G₂, F] [S, A, B, D, G₁, C]
 7 E [G₂, F, G₃] [S, A, B, D, G₁, C, E]
 8 G₂ [F, G₃] [S, A, B, D, G₁, C, E, G₂]
 9 F [G₃] [S, A, B, D, G₁, C, E, G₂, F]
 10 G₃ [] [S, A, B, D, G₁, C, E, G₂, F, G₃]

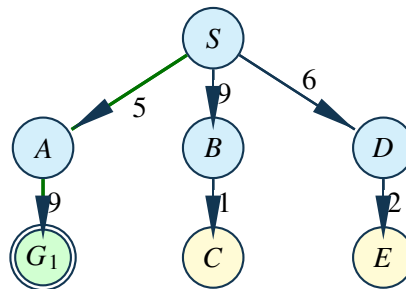
Operações relevantes:

- Passo 1, S: Inserir A, B, D.
- Passo 2, A: Inserir G₁. Ignorar B: já em OPEN.
- Passo 3, B: Inserir C. Ignorar A: já em VISITED.
- Passo 4, D: Inserir E. Ignorar C: já em OPEN. Ignorar S: já em VISITED.
- Passo 5, G₁: Objetivo confirmado; continuar se faltarem objetivos.
- Passo 6, C: Inserir G₂, F. Ignorar S: já em VISITED.
- Passo 7, E: Inserir G₃.
- Passo 8, G₂: Objetivo confirmado; continuar se faltarem objetivos.
- Passo 9, F: Ignorar D: já em VISITED. Ignorar G₃: já em OPEN.
- Passo 10, G₃: Objetivo confirmado; continuar se faltarem objetivos.

Ordem de retirada: [S, A, B, D, G₁, C, E, G₂, F, G₃].

Ordem de expansão (excluindo objetivos): [S, A, B, D, C, E, F].

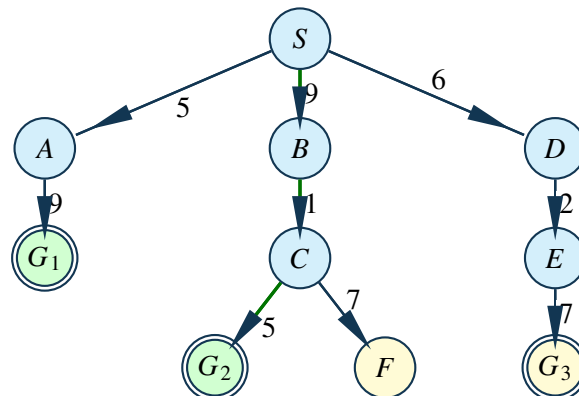
Ordem dos objetivos: G₁ → G₂ → G₃. **BFS — ao retirar G₁**



Amarelo: em OPEN; azul: visitado; verde: objetivo confirmado. As ligações verdes mostram o caminho até ao objetivo atual. A árvore contém apenas as descobertas aceites.

$$\boxed{S \rightarrow A \rightarrow G_1} \quad d = 2, \quad g = 5 + 9 = 14.$$

BFS — ao retirar G₂

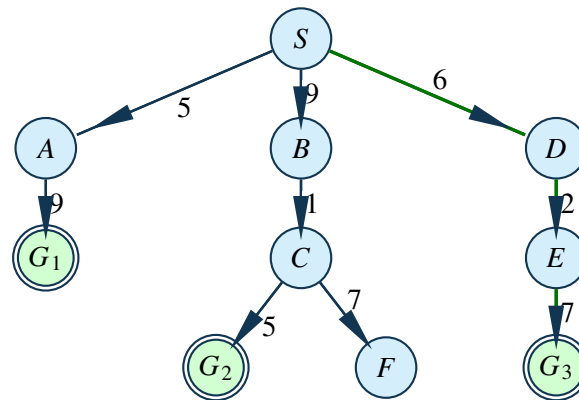


Amarelo: em OPEN; azul: visitado; verde: objetivo confirmado. As ligações verdes mostram o

caminho até ao objetivo atual. A árvore contém apenas as descobertas aceites.

$$\boxed{S \rightarrow B \rightarrow C \rightarrow G_2} \quad d = 3, \quad g = 9 + 1 + 5 = 15.$$

BFS — ao retirar G_3



Amarelo: em OPEN; azul: visitado; verde: objetivo confirmado. As ligações verdes mostram o caminho até ao objetivo atual. A árvore contém apenas as descobertas aceites.

$$\boxed{S \rightarrow D \rightarrow E \rightarrow G_3} \quad d = 3, \quad g = 6 + 2 + 7 = 15.$$

Execução da DFS

@p.05p.09p.37p.42@

Passo Retirado OPEN após o passo VISITED

```

0 - [ S ] [ ]
1 S [ A, B, D ] [ S ]
2 A [ G1, B, D ] [ S, A ]
3 G1 [ B, D ] [ S, A, G1 ]
4 B [ C, D ] [ S, A, G1, B ]
5 C [ G2, F, D ] [ S, A, G1, B, C ]
6 G2 [ F, D ] [ S, A, G1, B, C, G2 ]
7 F [ G3, D ] [ S, A, G1, B, C, G2, F ]
8 G3 [ D ] [ S, A, G1, B, C, G2, F, G3 ]

```

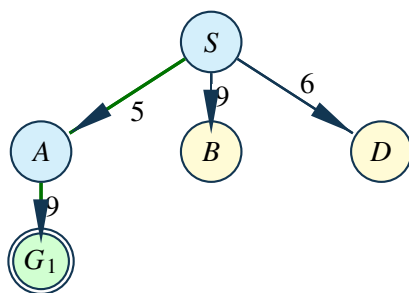
Operações relevantes:

- Passo 1, S: Inserir A, B, D.
- Passo 2, A: Inserir G_1 . Ignorar B: já em OPEN.
- Passo 3, G_1 : Objetivo confirmado; continuar se faltarem objetivos.
- Passo 4, B: Inserir C. Ignorar A: já em VISITED.
- Passo 5, C: Inserir G_2 , F. Ignorar S: já em VISITED.
- Passo 6, G_2 : Objetivo confirmado; continuar se faltarem objetivos.
- Passo 7, F: Inserir G_3 . Ignorar D: já em OPEN.
- Passo 8, G_3 : Objetivo confirmado; continuar se faltarem objetivos.

Ordem de retirada: [S, A, G_1 , B, C, G_2 , F, G_3].

Ordem de expansão (excluindo objetivos): [S, A, B, C, F].

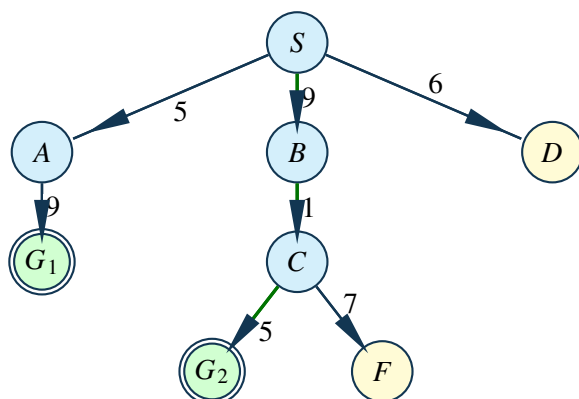
Ordem dos objetivos: $G_1 \rightarrow G_2 \rightarrow G_3$. **DFS — ao retirar G_1**



Amarelo: em OPEN; azul: visitado; verde: objetivo confirmado. As ligações verdes mostram o caminho até ao objetivo atual. A árvore contém apenas as descobertas aceites.

$$\boxed{S \rightarrow A \rightarrow G_1} \quad d = 2, \quad g = 5 + 9 = 14.$$

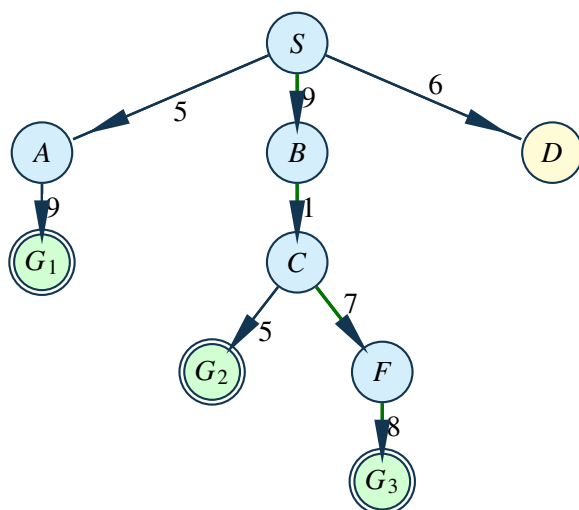
DFS — ao retirar G_2



Amarelo: em OPEN; azul: visitado; verde: objetivo confirmado. As ligações verdes mostram o caminho até ao objetivo atual. A árvore contém apenas as descobertas aceites.

$$\boxed{S \rightarrow B \rightarrow C \rightarrow G_2} \quad d = 3, \quad g = 9 + 1 + 5 = 15.$$

DFS — ao retirar G_3



Amarelo: em OPEN; azul: visitado; verde: objetivo confirmado. As ligações verdes mostram o caminho até ao objetivo atual. A árvore contém apenas as descobertas aceites.

$$\boxed{S \rightarrow B \rightarrow C \rightarrow F \rightarrow G_3} \quad d = 4, \quad g = 9 + 1 + 7 + 8 = 25.$$

Execução da UCS

@p.05p.09p.37p.42@

Passo Retirado OPEN após o passo VISITED

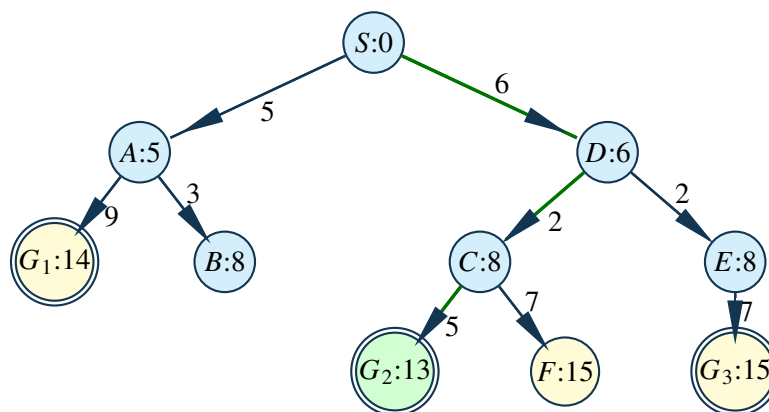
```

0 - [ S:0 ] [ ]
1 S : 0 [ A:5, D:6, B:9 ] [ S:0 ]
2 A : 5 [ D:6, B:8, G1:14 ] [ S:0, A:5 ]
3 D : 6 [ B:8, C:8, E:8, G1:14 ] [ S:0, A:5, D:6 ]
4 B : 8 [ C:8, E:8, G1:14 ] [ S:0, A:5, D:6, B:8 ]
5 C : 8 [ E:8, G2:13, G1:14, F:15 ] [ S:0, A:5, D:6, B:8, C:8 ]
6 E : 8 [ G2:13, G1:14, F:15, G3:15 ] [ S:0, A:5, D:6, B:8, C:8, E:8 ]
7 G2 : 13 [ G1:14, F:15, G3:15 ] [ S:0, A:5, D:6, B:8, C:8, E:8, G2:13 ]
8 G1 : 14 [ F:15, G3:15 ] [ S:0, A:5, D:6, B:8, C:8, E:8, G2:13, G1:14 ]
9 F : 15 [ G3:15 ] [ S:0, A:5, D:6, B:8, C:8, E:8, G2:13, G1:14, F:15 ]
10 G3 : 15 [ ] [ S:0, A:5, D:6, B:8, C:8, E:8, G2:13, G1:14, F:15, G3:15 ]

```

Operações relevantes:

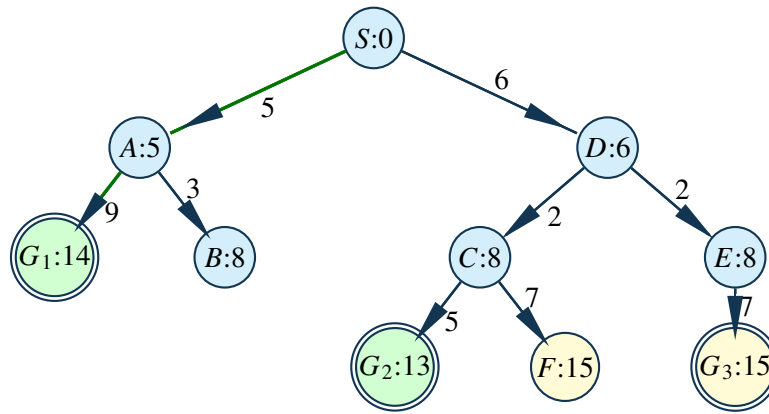
- Passo 1, S: Inserir A para 5. Inserir B para 9. Inserir D para 6.
- Passo 2, A: Inserir G₁ para 14. Melhorar B para 8.
- Passo 3, D: Inserir C para 8. Ignorar S: 7 não melhora 0. Inserir E para 8.
- Passo 4, B: Ignorar A: 10 não melhora 5. Ignorar C: 9 não melhora 8.
- Passo 5, C: Inserir G₂ para 13. Ignorar S: 14 não melhora 0. Inserir F para 15.
- Passo 6, E: Inserir G₃ para 15.
- Passo 7, G₂: Objetivo confirmado; continuar se faltarem objetivos.
- Passo 8, G₁: Objetivo confirmado; continuar se faltarem objetivos.
- Passo 9, F: Ignorar D: 17 não melhora 6. Ignorar G₃: 23 não melhora 15.
- Passo 10, G₃: Objetivo confirmado; continuar se faltarem objetivos.

Ordem de retirada: [S, A, D, B, C, E, G₂, G₁, F, G₃].**Ordem de expansão (excluindo objetivos):** [S, A, D, B, C, E, F].**Ordem dos objetivos:** G₂ → G₁ → G₃. **UCS — ao retirar G₂**

Amarelo: em OPEN; azul: visitado; verde: objetivo confirmado. As ligações verdes mostram o caminho até ao objetivo atual. A árvore contém apenas as descobertas aceites e os melhores pais conhecidos.

$$S \rightarrow D \rightarrow C \rightarrow G_2 \quad d = 3, \quad g = 6 + 2 + 5 = 13.$$

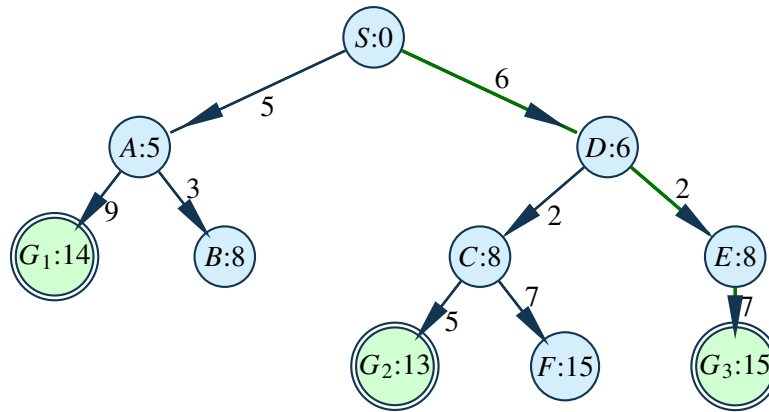
UCS — ao retirar G₁



Amarelo: em OPEN; azul: visitado; verde: objetivo confirmado. As ligações verdes mostram o caminho até ao objetivo atual. A árvore contém apenas as descobertas aceites e os melhores pais conhecidos.

$$S \rightarrow A \rightarrow G_1 \quad d = 2, \quad g = 5 + 9 = 14.$$

UCS — ao retirar G_3



Amarelo: em OPEN; azul: visitado; verde: objetivo confirmado. As ligações verdes mostram o caminho até ao objetivo atual. A árvore contém apenas as descobertas aceites e os melhores pais conhecidos.

$$S \rightarrow D \rightarrow E \rightarrow G_3 \quad d = 3, \quad g = 6 + 2 + 7 = 15.$$

Comparação dos resultados

Algoritmo	Objetivo	Caminho	Profundidade	Custo
BFS	G_1	$S \rightarrow A \rightarrow G_1$	2	14
BFS	G_2	$S \rightarrow B \rightarrow C \rightarrow G_2$	3	15
BFS	G_3	$S \rightarrow D \rightarrow E \rightarrow G_3$	3	15
DFS	G_1	$S \rightarrow A \rightarrow G_1$	2	14
DFS	G_2	$S \rightarrow B \rightarrow C \rightarrow G_2$	3	15
DFS	G_3	$S \rightarrow B \rightarrow C \rightarrow F \rightarrow G_3$	4	25
UCS	G_1	$S \rightarrow A \rightarrow G_1$	2	14
UCS	G_2	$S \rightarrow D \rightarrow C \rightarrow G_2$	3	13
UCS	G_3	$S \rightarrow D \rightarrow E \rightarrow G_3$	3	15

BFS e DFS confirmam os objetivos na ordem G_1, G_2, G_3 ; UCS confirma-os na ordem G_2, G_1, G_3 , com custos 13, 14 e 15.

Na DFS, ao expandir A , B já está em OPEN: não se cria uma entrada B com pai A . Assim, B

conserva o pai S . Ao expandir F , D também já está em OPEN; apenas G_3 é inserido. A pesquisa termina ao retirar G_3 , deixando D na pilha e E ainda por descobrir.

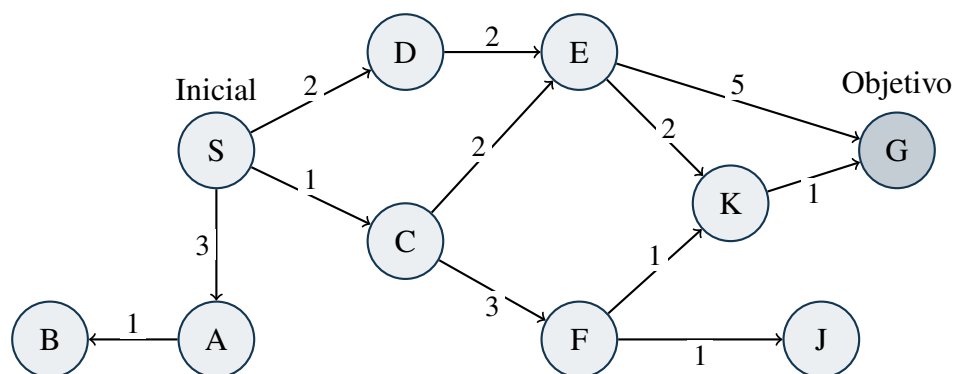
Na UCS, B passa de custo 9 para 8, com pai A , enquanto está em OPEN. Não se trata de reabrir um nó visitado. Já C conserva o pai D e o custo 8: o caminho via B custaria 9. No final, F e G_3 têm custo 15; F sai primeiro por ter sido inserido antes.

BFS encontra caminhos de menor profundidade para cada objetivo. DFS não garante menor profundidade nem menor custo; neste exemplo, o caminho para G_3 tem profundidade 4 e custo 25. UCS encontra os caminhos de menor custo. O valor comparado é sempre a soma dos custos desde S .

A ordem de retirada não é um caminho: dois estados retirados consecutivamente podem não estar ligados. Os caminhos são reconstruídos seguindo os pais. VISITED e a verificação de OPEN evitam repetições na BFS e na DFS; na UCS, os melhores custos permitem rejeitar caminhos piores e atualizar caminhos melhores.

PROCURA NUM GRAFO: BFS, DFS E UCS

Considere o grafo orientado seguinte. O estado inicial é S e o estado objetivo é G . Os números nas ligações representam os custos das ações. Só é permitido percorrer cada ligação no sentido da seta.



O que se pede

- Execute BFS, DFS e UCS desde S até G . Para cada algoritmo, apresente a ordem de retirada dos nós e a evolução da fronteira.
- Reconstrua o caminho encontrado por cada algoritmo. Calcule a sua profundidade e o seu custo total.
- Compare os resultados: que algoritmo encontra o caminho de menor custo? Esse caminho tem também o menor número de ações?

Solução proposta

a) Execução da BFS

A fila é apresentada com o próximo nó a retirar à esquerda. Inicialmente, S é inserido e marcado como alcançado.

Operação	Fila	Novos nós
Inicial	[S]	S
Retirar e expandir S	[A, C, D]	A, C, D
Retirar e expandir A	[C, D, B]	B
Retirar e expandir C	[D, B, E, F]	E, F
Retirar e expandir D	[B, E, F]	Nenhum
Retirar e expandir B	[E, F]	Nenhum
Retirar e expandir E	[F, G, K]	G, K
Retirar e expandir F	[G, K, J]	J
Retirar G	[K, J]	Objetivo: terminar

Ao expandir D, E já foi alcançado através de C. Ao expandir F, K já foi alcançado através de E. Esses estados não são reinseridos.

A ordem de retirada é:

$$S, A, C, D, B, E, F, G.$$

Os pais relevantes para a solução são:

$$\text{pai}(G) = E, \quad \text{pai}(E) = C, \quad \text{pai}(C) = S.$$

Logo, o caminho encontrado é:

$$S \longrightarrow C \longrightarrow E \longrightarrow G.$$

A profundidade e o custo são:

$$d = 3, \quad g(G) = 1 + 2 + 5 = \boxed{8}.$$

Foram retirados 8 nós e expandidos 7. K e J permanecem na fila quando a procura termina.

Execução da DFS

O topo da pilha está à direita. Os sucessores novos são inseridos por ordem alfabética inversa.

Operação	Pilha	Inserção
Inicial	[S]	S
Retirar e expandir S	[D, C, A]	D, C, A
Retirar e expandir A	[D, C, B]	B
Retirar e expandir B	[D, C]	Nenhum
Retirar e expandir C	[D, F, E]	F, E
Retirar e expandir E	[D, F, K, G]	K, G
Retirar G	[D, F, K]	Objetivo: terminar

A procura começa pelo ramo $S \rightarrow A \rightarrow B$. Como B não tem sucessores, retira-se a alternativa C, que estava pendente na pilha.

Esta continuação da procura não representa uma deslocação de B para C: não existe essa ligação no grafo.

A ordem de retirada é:

$$S, A, B, C, E, G.$$

Os pais do objetivo conduzem ao caminho:

$$S \longrightarrow C \longrightarrow E \longrightarrow G.$$

A profundidade e o custo são:

$$d = 3, \quad g(G) = 1 + 2 + 5 = \boxed{8}.$$

Foram retirados 6 nós e expandidos 5. D, F e K permanecem na pilha.

Execução da UCS — evolução da fronteira

A UCS retira sempre o nó da fronteira com menor custo acumulado desde S. Em empate, usamos a ordem alfabética.

A notação $E : 3$ significa que o melhor caminho conhecido até E tem custo 3. Para cada sucessor, calculamos:

$$g(\text{sucessor}) = g(\text{nó retirado}) + c(\text{ligação}).$$

A tabela mostra a fronteira **depois de cada operação**, ordenada pelo próximo nó a retirar.

Operação	Fronteira após a operação	O que acontece?
Inicializar	$[S : 0]$	Inserir S com custo 0.
Retirar e expandir S	$[C : 1, D : 2, A : 3]$	Inserir A com 3, C com 1 e D com 2. Os três têm pai S. O próximo é C, de menor custo.
Retirar e expandir C	$[D : 2, A : 3, E : 3, F : 4]$	Inserir E com $1 + 2 = 3$ e F com $1 + 3 = 4$. Ambos têm pai C. O próximo é D, que já estava na fronteira.
Retirar e expandir D	$[A : 3, E : 3, F : 4]$	Por D, chegar a E custaria $2 + 2 = 4$. Como E já tem custo 3, não atualizar. A sai antes de E por ordem alfabética.
Retirar e expandir A	$[E : 3, B : 4, F : 4]$	Inserir B com $3 + 1 = 4$ e pai A.
Retirar e expandir E	$[B : 4, F : 4, K : 5, G : 8]$	Inserir K com $3 + 2 = 5$ e G com $3 + 5 = 8$, ambos com pai E. G foi gerado, mas ainda não é retirado.
Retirar e expandir B	$[F : 4, K : 5, G : 8]$	B não tem sucessores. B saiu antes de F pelo desempate alfabético.
Retirar e expandir F	$[J : 5, K : 5, G : 8]$	Inserir J com $4 + 1 = 5$ e pai F. Por F, K também custaria 5: manter o custo e o pai anteriores.
Retirar e expandir J	$[K : 5, G : 8]$	J não tem sucessores. J saiu antes de K pelo desempate alfabético.
Retirar e expandir K	$[G : 6]$	Por K, chegar a G custa $5 + 1 = 6$. Como $6 < 8$, atualizar G: custo 6 e pai K.
Retirar G	$[]$	G é o objetivo. Terminar sem o expandir.

Caminho encontrado: seguindo os pais desde G e invertendo a sequência, obtemos:

$$S \rightarrow C \rightarrow E \rightarrow K \rightarrow G.$$

$$\text{Custo total} = 1 + 2 + 2 + 1 = 6, \quad \text{profundidade} = 4 \text{ ações.}$$

Ordem de retirada:

$$S, C, D, A, E, B, F, J, K, G.$$

Foram retirados **10 nós** e expandidos **9**.

O ponto essencial: G foi inicialmente encontrado com custo 8, mas a procura continuou e melhorou esse custo para 6. A UCS termina ao **retirar o objetivo**, não ao gerá-lo. Com custos não negativos e atualização correta dos melhores custos, o caminho devolvido tem custo mínimo.

c) Comparar os resultados

A **BFS** encontra uma solução com o menor número de ações. Contudo, os custos das ações são diferentes, pelo que essa solução não tem necessariamente o menor custo.

A **DFS** encontra, nesta execução, o mesmo caminho da BFS. Isso depende da ordem de exploração e não constitui uma garantia de menor profundidade ou menor custo.

A **UCS** encontra uma solução de custo mínimo. Neste caso, realiza mais ações, mas evita a ligação $E \rightarrow G$, de custo 5, substituindo-a por:

$$E \rightarrow K \rightarrow G, \quad 2 + 1 = 3.$$

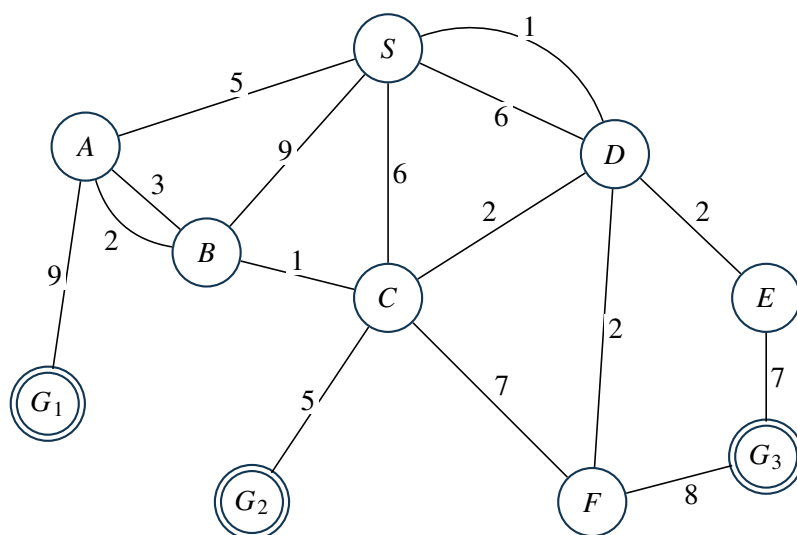
Algoritmo	Nós retirados	Nós expandidos
BFS	8	7
DFS	6	5
UCS	10	9

O objetivo é contado como retirado, mas não como expandido.

Conclusão: neste grafo, um caminho com quatro ações tem custo menor do que o caminho com três ações encontrado pela BFS. A profundidade conta ações; o custo soma os valores das ligações.

PROCURA NUM GRAFO: BFS, DFS E UCS

Considere o grafo dirigido e ponderado abaixo. O estado inicial é S e os estados objetivo são G_1 , G_2 e G_3 . Os números junto dos arcos indicam os respetivos custos. Pretende-se encontrar um caminho de S até qualquer objetivo.

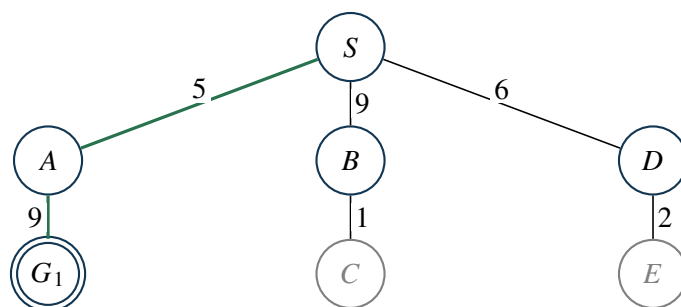


O que se pede

1. Aplique BFS, DFS e UCS, usando pesquisa em grafo e as convenções indicadas abaixo.
2. Para cada algoritmo, desenhe a árvore de pesquisa até à paragem e preencha, passo a passo, as estruturas OPEN/Fronteira e VISITED.
3. Indique a ordem de visita, a ordem de expansão, o objetivo encontrado, o caminho solução, a profundidade e o custo total.
4. Compare as soluções: qual tem menor profundidade e qual tem menor custo? Explique o papel de VISITED na prevenção de ciclos.

Solução proposta

a) Execução da BFS



Passo	Nó retirado	OPEN após o passo	VISITED
0	–	[S]	[]
1	S	[A, B, D]	[S]
2	A	[B, D, G ₁]	[S, A]
3	B	[D, G ₁ , C]	[S, A, B]
4	D	[G ₁ , C, E]	[S, A, B, D]
5	G ₁	[C, E]	[S, A, B, D, G ₁]

Tratamento de repetições

- Ao expandir A, o sucessor B já está em OPEN; não se insere uma segunda entrada.
- Ao expandir B, o sucessor A já está em VISITED.

- Ao expandir D , C já está em OPEN e S já está em VISITED; só se acrescenta E .
- No passo 5, G_1 é retirado e reconhecido como objetivo. C e E ficam pendentes.

b) Caminho, ordem de pesquisa e custo

Ordem de visita: S, A, B, D, G_1 .

Ordem de expansão: S, A, B, D .

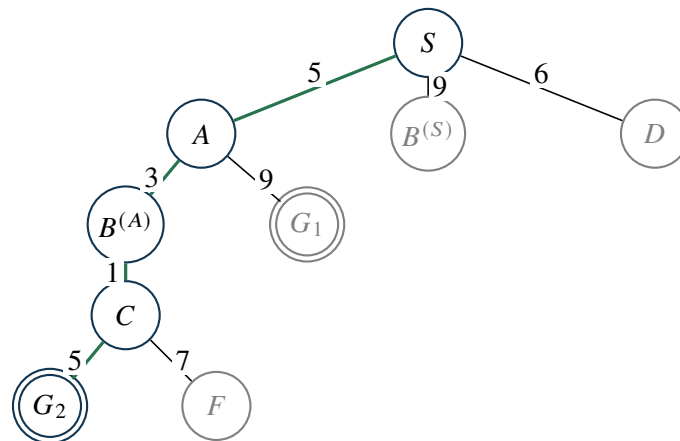
Caminho solução: $S \rightarrow A \rightarrow G_1$.

Profundidade: 2.

Custo: $5 + 9 = 14$.

BFS encontra um objetivo à menor profundidade, porque visita os estados por níveis. Os custos dos arcos não intervêm na escolha da próxima entrada. Neste grafo, a solução de menor profundidade não é a solução de menor custo.

Execução da DFS



$B^{(S)}$ e $B^{(A)}$ são duas entradas do *mesmo estado* B , descobertas respetivamente a partir de S e de A . O sobrescrito identifica o pai da entrada; não identifica estados novos.

Passo	Nó retirado	OPEN após o passo	VISITED
0	–	[S]	[]
1	S	[$A, B^{(S)}, D$]	[S]
2	A	[$B^{(A)}, G_1, B^{(S)}, D$]	[S, A]
3	$B^{(A)}$	[$C, G_1, B^{(S)}, D$]	[S, A, B]
4	C	[$G_2, F, G_1, B^{(S)}, D$]	[S, A, B, C]
5	G_2	[$F, G_1, B^{(S)}, D$]	[S, A, B, C, G_2]

Ao expandir B , rejeita-se A , já visitado. Ao expandir C , rejeita-se S . A entrada antiga $B^{(S)}$ fica na pilha; se fosse retirada posteriormente, seria descartada, pois B já pertence a VISITED. Não há uma segunda expansão de B .

Ordem de visita: S, A, B, C, G_2 .

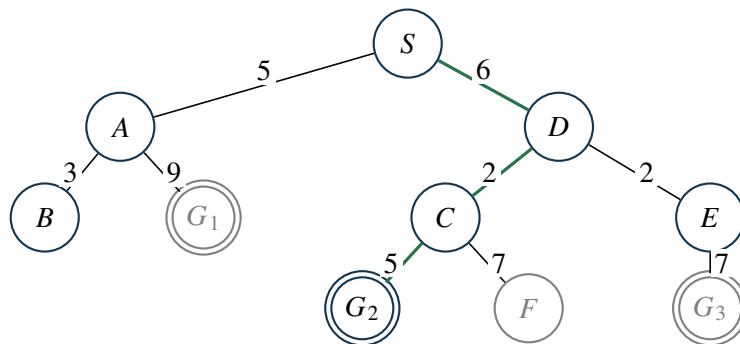
Ordem de expansão: S, A, B, C .

Caminho solução: $S \rightarrow A \rightarrow B \rightarrow C \rightarrow G_2$.

Profundidade e custo: $d = 4$, $5 + 3 + 1 + 5 = 14$.

A convenção importa. Se se proibissem também entradas repetidas em OPEN neste DFS com pilha, $A \rightarrow B$ seria rejeitado e obter-se-ia $S \rightarrow A \rightarrow G_1$. A variante aqui definida conserva a exploração em profundidade usada na conversa. DFS não garante menor profundidade nem menor custo.

Execução da UCS



A árvore mostra os melhores pais conhecidos no momento da paragem. A entrada inicial $S \rightarrow B$, com custo 9, foi substituída por $S \rightarrow A \rightarrow B$, com custo 8. Na tabela, n_c significa estado n com custo acumulado $g(n) = c$.

Passo	Retirado	OPEN após o passo	VISITED
0	–	[S_0]	[]
1	S_0	[A_5, D_6, B_9]	[S]
2	A_5	[$D_6, B_8, (G_1)_{14}$]	[S, A]
3	D_6	[$B_8, C_8, E_8, (G_1)_{14}$]	[S, A, D]
4	B_8	[$C_8, E_8, (G_1)_{14}$]	[S, A, D, B]
5	C_8	[$E_8, (G_2)_{13}, (G_1)_{14}, F_{15}$]	[S, A, D, B, C]
6	E_8	[$(G_2)_{13}, (G_1)_{14}, F_{15}, (G_3)_{15}$]	[S, A, D, B, C, E]
7	$(G_2)_{13}$	[$(G_1)_{14}, F_{15}, (G_3)_{15}$]	[S, A, D, B, C, E, G_2]

Em A, melhora-se B: $5 + 3 = 8 < 9$. Em D, descobrem-se C e E, ambos com custo 8; B sai primeiro por ter sido inserido antes. Em B, o caminho para C custa $8 + 1 = 9$, pelo que se conserva $g(C) = 8$. Os regressos a A e S são rejeitados. O objetivo é aceite ao sair de OPEN, não quando é descoberto.

Ordem de visita: S, A, D, B, C, E, G_2 .

Ordem de expansão: S, A, D, B, C, E .

Solução: $S \rightarrow D \rightarrow C \rightarrow G_2$, $d = 3$, $6 + 2 + 5 = 13$.

c) Comparar os resultados

Método	Caminho solução	Profundidade	Custo
BFS	$S \rightarrow A \rightarrow G_1$	2	14
DFS	$S \rightarrow A \rightarrow B \rightarrow C \rightarrow G_2$	4	14
UCS	$S \rightarrow D \rightarrow C \rightarrow G_2$	3	13

BFS minimiza o número de arcos; UCS minimiza o custo, neste grafo de custos positivos. VISITED impede expansões repetidas e a exploração indefinida de ciclos como $A \rightarrow B \rightarrow A$ e $S \rightarrow D \rightarrow S$.

ORDENAÇÃO POR TROCAS

Pretende-se ordenar a lista $(2, 3, 1)$ por ordem crescente. A única ação permitida é trocar os elementos de duas posições diferentes. Pode trocar-se qualquer par de posições, mesmo que não sejam adjacentes. Cada troca custa 1.

O que se pede

- Defina o estado inicial, a representação dos estados, as ações, a transição e o teste de objetivo.
- Enumere os estados possíveis e apresente os sucessores de cada estado para as trocas das posições $(1, 2)$, $(1, 3)$ e $(2, 3)$.
- Determine o número mínimo de trocas e apresente um caminho ótimo. Justifique por que motivo o BFS é adequado e por que é útil controlar os estados repetidos.

Solução proposta

a) Componentes do problema

Um estado é a lista na sua ordem atual:

$$s = (x_1, x_2, x_3).$$

A posição faz parte da representação: $(2, 3, 1)$ e $(1, 2, 3)$ são estados diferentes, embora contenham os mesmos números.

O estado inicial é $(2, 3, 1)$ e o objetivo é $(1, 2, 3)$. As três ações são trocar as posições 1 e 2, 1 e 3, ou 2 e 3. A transição troca os dois elementos escolhidos e conserva o restante. Cada ação tem custo 1.

b) Espaço de estados e transições

Existem três escolhas para a primeira posição, duas para a segunda e uma para a última. Logo, existem $3 \times 2 \times 1 = 6$ estados.

Estado	Trocar 1 e 2	Trocar 1 e 3	Trocar 2 e 3
$(1, 2, 3)$	$(2, 1, 3)$	$(3, 2, 1)$	$(1, 3, 2)$
$(1, 3, 2)$	$(3, 1, 2)$	$(2, 3, 1)$	$(1, 2, 3)$
$(2, 1, 3)$	$(1, 2, 3)$	$(3, 1, 2)$	$(2, 3, 1)$
$(2, 3, 1)$	$(3, 2, 1)$	$(1, 3, 2)$	$(2, 1, 3)$
$(3, 1, 2)$	$(1, 3, 2)$	$(2, 1, 3)$	$(3, 2, 1)$
$(3, 2, 1)$	$(2, 3, 1)$	$(1, 2, 3)$	$(3, 1, 2)$

A tabela representa as ligações do grafo. Cada troca é reversível: repetir a mesma ação recupera o estado anterior.

c) Solução ótima

Uma solução é:

$$(2, 3, 1) \xrightarrow{\text{trocar 1 e 3}} (1, 3, 2) \xrightarrow{\text{trocar 2 e 3}} (1, 2, 3).$$

O custo é $1 + 1 = 2$.

Não existe uma solução de custo 0, pois a lista inicial não está ordenada. Também não existe solução com uma troca: nenhum dos três sucessores de $(2, 3, 1)$ é $(1, 2, 3)$. Como existe solução com duas

trocas, esse é o mínimo.

O BFS explora por profundidade crescente. Com custos unitários, o primeiro objetivo retirado da fila tem custo mínimo.

Controlar os estados repetidos evita voltar a explorar listas já alcançadas. Sem esse controle, as trocas reversíveis originam ciclos e múltiplos caminhos para o mesmo estado. Um estado pode aparecer em vários nós da árvore de procura, cada um com o seu caminho.

MUNDO DO ASPIRADOR: BFS E DFS

Um aspirador desloca-se entre duas divisões, E (esquerda) e D (direita). Cada divisão está limpa ou suja. Pretende-se limpar ambas, independentemente da posição final do aspirador.

Representa-se um estado por (p, s_E, s_D) : p indica a posição e cada indicador de sujidade vale 1 se a divisão estiver suja e 0 se estiver limpa. Use os seguintes identificadores:

Identificador	Estado	Identificador	Estado
1	$(E, 1, 1)$	5	$(E, 1, 0)$
2	$(D, 1, 1)$	6	$(D, 1, 0)$
3	$(E, 0, 1)$	7	$(E, 0, 0)$
4	$(D, 0, 1)$	8	$(D, 0, 0)$

O estado inicial é 1. Existem três ações, geradas sempre pela ordem **A, D, E**: A aspira a divisão atual; D move para a direita; E move para a esquerda. Aspirar uma divisão limpa ou mover para a divisão onde já se encontra mantém o estado. Cada ação custa 1.

Regras de execução

Use procura em grafo. Registe um estado como alcançado logo que é inserido na fronteira e não o insira novamente. O estado inicial é registado antes de começar.

No BFS, insira no fim e retire do início da fila. No DFS, insira e retire no topo da pilha, situado à direita. Como os sucessores são inseridos pela ordem A, D, E, o último sucessor novo inserido será o primeiro a sair no DFS.

Teste o objetivo **depois de retirar** um nó. O objetivo é retirado, mas não expandido. Na apresentação da fronteira, use os identificadores numéricos dos estados.

O que se pede

- Indique os estados objetivo e os sucessores do estado 1.
- Execute BFS e DFS até ao primeiro objetivo. Registe a fronteira após cada operação e reconstrua os caminhos através dos pais.
- Compare o custo dos caminhos, o número de nós retirados e o número de nós expandidos. O exemplo prova que uma estratégia é sempre melhor?

Solução proposta

a) Objetivos e primeiros sucessores

Os objetivos são 7 e 8, pois ambos têm $s_E = s_D = 0$.

A partir de 1, a ação A produz 3; D produz 2; E mantém 1. Como 1 já foi alcançado, só 3 e 2 são

inseridos, por essa ordem. Registam-se os pais $pai(3) = 1$ e $pai(2) = 1$.

b) BFS: execução da fila

Na fila, o próximo nó a retirar está à esquerda.

Operação	Fila	Explicação
Inicial	[1]	O estado 1 já está registado.
Expandir 1	[3, 2]	Inserir 3 e 2. O sucessor 1 é repetido.
Expandir 3	[2, 4]	A e E mantêm 3; D gera 4. Registrar $pai(4) = 3$.
Expandir 2	[4, 6]	A gera 6; D mantém 2; E gera 1. Só 6 é novo: $pai(6) = 2$.
Expandir 4	[6, 8]	A gera 8; os outros sucessores já foram alcançados. $pai(8) = 4$.
Expandir 6	[8, 5]	E gera 5; A e D mantêm 6. $pai(5) = 6$.
Retirar 8	[5]	É objetivo: terminar sem o expandir.

Embora 8 seja gerado ao expandir 4, o algoritmo só termina quando o retira. Antes disso, retira e expande 6, que estava à frente na fila.

Seguindo os pais, obtemos $8 \leftarrow 4 \leftarrow 3 \leftarrow 1$. Invertendo a sequência:

$$1 \xrightarrow{A} 3 \xrightarrow{D} 4 \xrightarrow{A} 8.$$

A solução aspira à esquerda, desloca-se para a direita e aspira aí. Custo: 3.

DFS: execução da pilha

O topo da pilha está à direita.

Operação	Pilha	Explicação
Inicial	[1]	Começar com o estado inicial.
Expandir 1	[3, 2]	Inserir primeiro 3 e depois 2. O topo é 2.
Expandir 2	[3, 6]	Só 6 é novo. Registrar $pai(6) = 2$.
Expandir 6	[3, 5]	Só 5 é novo. Registrar $pai(5) = 6$.
Expandir 5	[3, 7]	A gera 7; D gera 6 e E mantém 5. Só 7 é novo: $pai(7) = 5$.
Retirar 7	[3]	É objetivo: terminar sem o expandir.

O caminho reconstruído é:

$$1 \xrightarrow{D} 2 \xrightarrow{A} 6 \xrightarrow{E} 5 \xrightarrow{A} 7.$$

O aspirador começa por ir para a direita, limpa-a, regressa à esquerda e limpa-a. Custo: 4. O estado 3 permanece pendente na pilha.

c) Comparação com o mesmo critério

Estratégia	Ordem de retirada	Custo	Retirados	Expandidos
BFS	1, 3, 2, 4, 6, 8	3	6	5
DFS	1, 2, 6, 5, 7	4	5	4

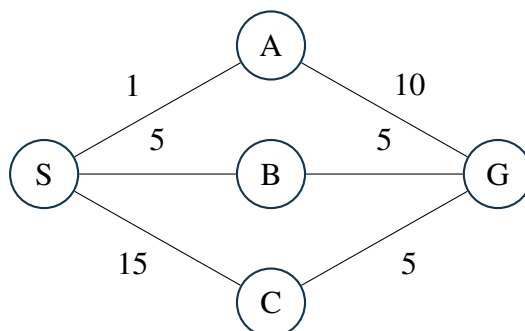
O objetivo conta como retirado, mas não como expandido. Um sucessor repetido que não entra na fronteira não acrescenta uma retirada.

O DFS expande menos nós nesta execução, mas devolve um caminho mais caro. O BFS encontra uma solução ótima: é necessário aspirar duas divisões e fazer pelo menos uma deslocação, num total mínimo de três ações.

Não existe uma superioridade geral demonstrada por este exemplo. A comparação depende do critério, da estrutura do problema e da ordem das ações. As garantias de optimalidade do BFS com custos unitários não se estendem ao DFS.

PROCURA DE CUSTO UNIFORME

Considere o grafo não orientado seguinte. Cada ligação pode ser percorrida nos dois sentidos e o número junto à aresta indica o custo da ação. O estado inicial é S e o objetivo é G. Os custos são dados didáticos.



Regras de execução

Use UCS com uma fronteira OPEN ordenada pelo menor custo acumulado g . Inicialmente, $g(S) = 0$ e os restantes custos valem ∞ (ainda não foi encontrado um caminho).

Gere os vizinhos por ordem alfabética. Para um sucessor m de n , calcule $g(n) + c(n, m)$. Se este valor for estritamente menor do que o melhor custo conhecido de m , atualize o custo, o pai e a entrada em OPEN. Não mantenha entradas duplicadas. Em igualdade de prioridades, use FIFO.

Teste o objetivo apenas quando o nó é retirado de OPEN.

O que se pede

- Execute UCS, mostrando OPEN após cada expansão, os custos e as atualizações dos pais.
- Quando A gera G com custo 11, já foi encontrada uma solução. Por que motivo não deve o algoritmo terminar nessa altura?
- Reconstrua a solução final, verifique o custo e conte os nós retirados e expandidos. Que condições justificam a optimalidade neste exemplo?

Solução proposta

a) Atualizar a fronteira pelo custo acumulado

A notação $A : 1$ significa que o melhor caminho conhecido até A tem custo 1.

Operação	OPEN	Atualizações
Inicial	$[S : 0]$	Só se conhece o caminho inicial.
Expandir S	$[A : 1, B : 5, C : 15]$	Pais de A, B e C: S.
Expandir A	$[B : 5, G : 11, C : 15]$	$g(G) = 1 + 10 = 11$; $pai(G) = A$.
Expandir B	$[G : 10, C : 15]$	Melhorar G: $5 + 5 = 10 < 11$; $pai(G) = B$.
Retirar G	$[C : 15]$	Objetivo: terminar sem expandir G.

Ao expandir A, também se considera regressar a S. O candidato tem custo $1 + 1 = 2$, superior a $g(S) = 0$, pelo que não há atualização.

Ao expandir B, regressar a S teria custo $5 + 5 = 10$, também sem melhoria. Já o caminho até G melhora de 11 para 10. A entrada de G é atualizada e o seu pai passa de A para B.

b) Distinguir gerar de retirar o objetivo

Gerar G através de A revela uma solução válida, mas não prova que seja a mais barata. Na fronteira, B ainda tem custo acumulado 5 e pode conduzir a um caminho melhor.

É precisamente o que acontece:

$$C(S, A, G) = 1 + 10 = 11, \quad C(S, B, G) = 5 + 5 = 10.$$

Terminar quando G é gerado devolveria a solução de custo 11, que não é ótima. O teste na retirada permite considerar primeiro os candidatos com menor prioridade de custo.

c) Reconstrução e verificação

No final, $pai(G) = B$ e $pai(B) = S$. Seguindo os pais obtemos $G \leftarrow B \leftarrow S$. O caminho, na ordem de execução, é:

$$\boxed{S \rightarrow B \rightarrow G}, \quad C = 5 + 5 = \boxed{10}.$$

As duas ligações existem, o último estado é G e o custo resulta da soma das arestas. A solução tem profundidade 2.

Foram retirados **4 nós**: S, A, B e G. Foram expandidos **3 nós**: S, A e B. C permanece na fronteira.

O grafo é finito, todos os custos são positivos e o algoritmo conserva as melhorias de custo. Quando G é retirado, tem o menor g de OPEN. Com custos não negativos, prolongar um caminho não reduz o seu custo acumulado. Com estas condições e a atualização correta, UCS garante uma solução de custo mínimo.

Neste grafo, também podemos confirmar os caminhos simples: por A custa 11, por B custa 10 e por C custa 20. Voltas adicionais têm custo positivo e não melhoram o resultado.